

ServiceNow Development

Project Documentation

1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident)
- **Team ID:** SWUID2026286057
- **Team Members:** List of team members and their roles.

S.No	Team Member	Role
1	Naveen.R	Requirement analysis and UI Policy configuration
2	Rudhran.S.N	Client Script development
3	Ramesh.M	Testing and quality checks
4	Vivilian Joshua.A	Documentation, diagrams and report preparation
5	Sarathi.S	Instance setup, Update Set management and demo

2. Project Overview

- **Purpose:** To improve the quality and consistency of data entered in the ServiceNow Incident form by using Client Scripts and UI Policies. The project makes important fields mandatory, visible or read-only based on conditions, and gives users instant on-screen guidance and validation before a record is saved, without a server round trip.
- **Features:** Key features and functionalities of the project are:
 - o UI Policy: makes On hold reason visible and mandatory when State is On Hold.
 - o UI Policy: makes Resolution code and Resolution notes mandatory when State is Resolved.
 - o UI Policy: makes Description mandatory for Priority 1 - Critical incidents.
 - o UI Policy: makes key fields read-only once an incident is Closed.
 - o Client Script (onLoad): shows a guidance message when a new incident is created.
 - o Client Script (onChange): shows field messages when Category or Impact is changed.
 - o Client Script (onSubmit): blocks submission of a Priority 1 incident with a very short description.

3. Architecture

- **Frontend:** The frontend is the ServiceNow platform UI (Incident form, opened from Incident > Create New). Client-side behaviour is added with UI Policies and Client Scripts, which run in the user's browser and use the g_form and g_user client APIs to read values, set mandatory / read-only / visible states and display messages.
- **Backend:** The backend is the ServiceNow server platform (Glide), which handles ACLs, existing Business Rules, and the Table API. This project adds no custom server-side code; it only relies on the platform to save the validated record.

- **Database:** Data is stored in the incident table (extends the task table) of the ServiceNow instance. The fields used by this project are:

Field (Label)	Column Name	Usage in Project
Short description	short_description	Always mandatory
Description	description	Mandatory for Priority 1; length check on submit
Category / Subcategory	category / subcategory	onChange field message
Impact / Urgency / Priority	impact / urgency / priority	Priority 1 conditions and messages
State	state	Drives UI Policy conditions
On hold reason	hold_reason	Visible and mandatory when On Hold
Resolution code / notes	close_code / close_notes	Mandatory when Resolved

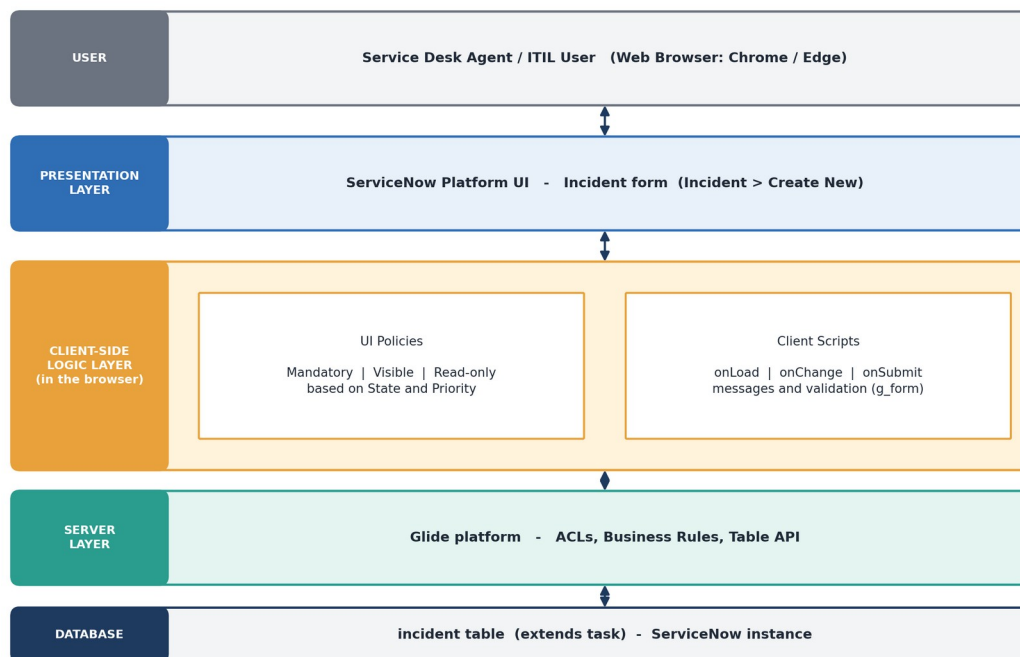


Figure: Solution Architecture

4. Setup Instructions

- **Prerequisites:** Software and accounts required:
 - o A free ServiceNow Developer account (developer.servicenow.com).
 - o A Personal Developer Instance (PDI) with the admin role.
 - o A modern web browser such as Google Chrome or Microsoft Edge.
- **Installation:** Step-by-step guide to set up the environment:
 1. Sign up at the ServiceNow Developer Program and request a Personal Developer Instance.
 2. Log in to the instance URL (https://<instance-name>.service-now.com) as the admin user.

3. Create a Local Update Set named "Client Script and UI Policy (Incident)" and set it as Current so all changes are captured.
4. Open System UI > UI Policies and create the four Incident UI Policies.
5. Open System Definition > Client Scripts and create the four Incident Client Scripts.
6. Mark the Update Set as Complete, so it can be exported to another instance if needed.

5. Folder Structure

ServiceNow does not use project folders. The components are organised as records inside the instance, as shown below.

- **Client:** Client-side records in the Application Navigator:
 - o System UI > UI Policies (table: sys_ui_policy) - conditions and UI Policy Actions.
 - o System Definition > Client Scripts (table: sys_script_client) - onLoad, onChange and onSubmit scripts.
- **Server:** Server-side records the project depends on:
 - o System Definition > Tables - the incident table and its dictionary entries.
 - o System Security > Access Control (ACL) - controls who can read and write incident fields.
 - o System Update Sets > Local Update Sets - tracks and moves the configuration.

6. Running the Application

- There is no local server to install; the platform runs in the cloud. Use the following steps to run and test the project.
 - o **Frontend:** Open the instance URL, then go to Incident > Create New (or Incident > All to open an existing record). The Client Scripts and UI Policies run automatically in the browser.
 - o **Backend:** No command is required. If the PDI is hibernating, wake it up from the developer portal and wait until the instance is available.

7. API Documentation

- The project is client-side, so no new endpoints were created. The Incident records are also reachable through the standard ServiceNow Table API:

Method	Endpoint	Description
GET	/api/now/table/incident	Retrieve a list of incidents. Parameters: sysparm_query, sysparm_limit, sysparm_fields
GET	/api/now/table/incident/{sys_id}	Retrieve a single incident
POST	/api/now/table/incident	Create an incident (JSON body)
PATCH	/api/now/table/incident/{sys_id}	Update fields of an incident
DELETE	/api/now/table/incident/{sys_id}	Delete an incident

- Example request body for POST /api/now/table/incident:

```
{ "short_description": "Laptop not booting", "category": "hardware",  
  "impact": "2", "urgency": "2" }
```
- Example response (trimmed):

```
{ "result": { "number": "INC0010001", "sys_id": "...", "state": "1",  
  "priority": "3" } }
```
- Note: Client Scripts and UI Policies run only in the browser UI. They do not run for REST API calls, so server-side rules would be needed to enforce the same checks there.

8. Authentication

- UI access uses a ServiceNow user ID and password session login. Enterprise instances can also use SSO or multi-factor authentication.
- API access uses Basic Authentication or OAuth 2.0 with a user account; the account's roles decide what it can do.
- Authorization is handled by roles (for example itil and admin) and ACLs on the incident table and its fields.
- Client-side checks such as `g_user.hasRole()` can be used in scripts, but they only improve usability; security is enforced by ACLs on the server.

9. User Interface

- Screenshots showcasing the different UI features of the project:

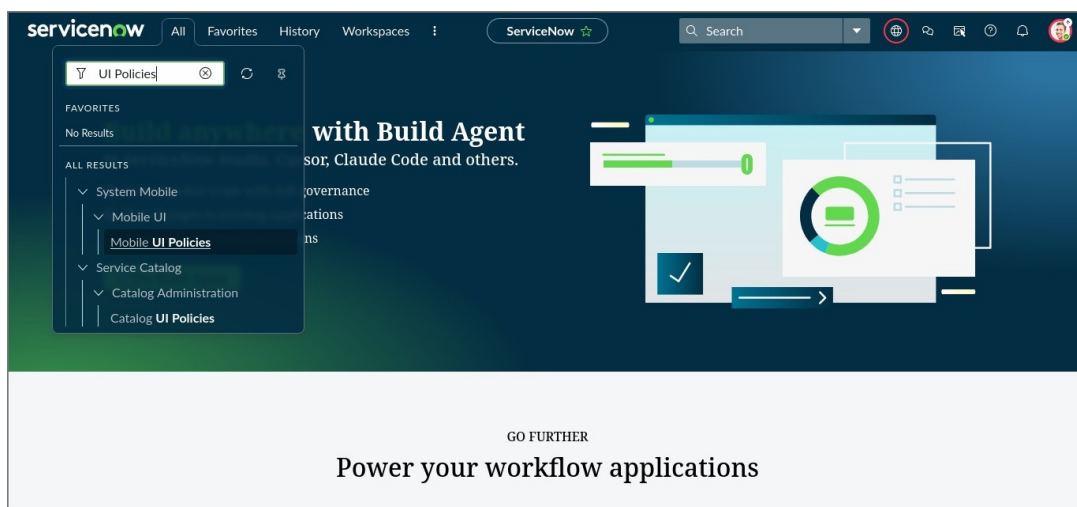


Figure 1: Opening UI Policies from the Application Navigator (All > UI Policies)

Short description	Table	Conditions	Reverse if false	On load	Updated	Order
High Impact Control	Incident [incident]	impact=1^EQ	true	true	2026-09-29 07:07:55	100
High Impact Control	Incident [incident]	impact=1^EQ	true	true	2026-09-28 23:55:49	100
Weekly Rotations Not All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true	2026-06-13 03:10:31	100
Weekly Rotations All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true	2026-06-13 03:10:31	100
No Reminders	Roster [cmn_rota_roster]	send_reminders=false^EQ	true	true	2026-06-13 03:10:31	100
Show pricing implications checkbox	Question [question]	type=8^ORtype=21^EQ	true	true	2026-06-13 02:56:32	100
Show price if checked	Question [question]	type=7^EQ	true	true	2026-06-13 02:56:32	100
Show Multiple Choices and Choice price	Item Option [sc_cat_item_option]	type=multiple_choice^ORtype=numeric_scal...	true	true	2026-06-13 02:56:32	100

Figure 2: UI Policies list filtered for "high impact" on the Incident table

10. Testing

- Testing strategy: manual functional testing of every UI Policy and Client Script on the Incident form, with a positive and a negative case for each.
- Tools used: ServiceNow Incident form, JavaScript Log and Field Watcher, browser Developer Tools (Console) and user impersonation to check role-based behaviour.

ID	Test Case	Expected Result
T1	Open a new incident	Guidance message is displayed at the top of the form
T2	Set State to On Hold	On hold reason appears and is mandatory; save is blocked if empty
T3	Set State to Resolved	Resolution code and Resolution notes become mandatory
T4	Set Priority 1 with description under 20 characters	Submit is blocked with an error message
T5	Change Category to Hardware	Field message appears on Subcategory
T6	Open a Closed incident	Key fields are read-only

11. Screenshots or Demo

- Provide screenshots or a link to a demo to showcase the application:
- **Demo Link:** [Paste project demo link here]

12. Known Issues

- Client Scripts and UI Policies do not run for REST API, import set or background updates, so data entered that way is not checked.
- Behaviour can differ in Service Operations Workspace or mobile apps unless the scripts are configured for those views.

- Client-side checks can be bypassed by technical users, so they must not replace server-side validation and ACLs.
- A PDI may hibernate after inactivity, which delays access to the instance.

13. Future Enhancements

- Add matching server-side Data Policies and Business Rules so the checks apply to API and import data too.
- Extend the same approach to Problem, Change Request and Service Catalog forms.
- Use Script Includes with GlideAjax for dynamic validation against other tables.
- Package the work as a scoped application and add Automated Test Framework (ATF) tests.